Guion video Prueba 2 · AuraSacrapp (7:00, sin audio)

El video **no lleva audio**: se reproduce en silencio y las dos personas relatan **en vivo** mientras corre.
Por eso el video se arma en escenas con tiempos fijos y el texto se ensaya con el cronómetro.
- **Persona A**: 0:00 a 3:30
- **Persona B**: 3:30 a 7:00
Si el reparto real del trabajo es distinto, intercambien los bloques. Lo importante es que cada quien hable de lo que programó. El profesor lo pide explícitamente.

Preparación de la grabación

1. Emulador **Android 16 (API 36) con Google Play**, internet activo y app recién instalada (desinstalar y volver a instalar, para que las fechas de los eventos sean frescas).
2. Grabar con la grabadora del emulador (**Extended controls a Screen record**) o con OBS/Xbox Game Bar. **Micrófono apagado.** Exportar a MP4.
3. Cargar en la cámara virtual del emulador `docs/qr/qr_01_capilla-del-buen-pastor.png` y `qr_99_codigo-no-valido.png` antes de grabar.
4. Cuentas (clave `Prueba123`): `visitante@aurasacrapp.cl`, `docente@aurasacrapp.cl`, `encargado.capilla@aurasacrapp.cl`.
5. Grabar **escena por escena** y unirlas (CapCut, Clipchamp, etc.), ajustando cada escena a su duración. El total debe ser **menor o igual a 7:00**.
6. Escena 1 (30 s): mostrar Android Studio con el árbol de paquetes. Es la única parte que no es la app.
7. Ensayar con el video puesto y cronómetro. Hablar a un ritmo tranquilo (unas 130 palabras por minuto).

PERSONA A · 0:00 a 3:30

Tiempo	Se ve en el video	Qué dice
0:00–0:30	Pantalla **Bienvenida** de la app	"Buenas tardes. Somos [nombres] y presentamos **AuraSacrapp**, la app móvil de la Red Patrimonial de Santiago, para el Área de Cultura del Arzobispado. Está hecha en **Kotlin, Jetpack Compose y Material Design 3**, con arquitectura **MVVM**. En esta entrega la app funciona con base de datos local Room y todavía no se conecta al backend, como pidió el profesor."
0:30–1:15	**Android Studio**: árbol de paquetes `cl.duoc.aurasacrapp`, recorriendo uno a uno	"Esta es la organización del proyecto. **model** tiene las entidades de Room: Comuna, Recinto, Usuario, Evento y Visita, más los resultados de consultas con JOIN. **dao** tiene las consultas. **database** tiene AppDatabase y los datos iniciales. **repository** es la única puerta de acceso a los datos. **viewmodel** guarda el estado de cada pantalla con StateFlow. **view** son las pantallas en Compose y **navigation** define las rutas y el NavHost. El flujo es siempre View, ViewModel, Repository, DAO y Room."
1:15–1:40	Archivo `build.gradle.kts`, sección de dependencias (zoom rápido)	"Las dependencias principales son: **Compose con Material 3**, **Navigation Compose** para moverse entre pantallas, **ViewModel Compose**, **Room con KSP** para la base de datos local, el **driver SQLite incluido**, los **íconos extendidos** y **ML Kit Code Scanner** para leer QR."
1:40–2:10	**Registro**: mostrar el formulario, cambiar entre Visitante y Docente (aparece "Colegio"), tocar *Crear cuenta*, aparece el aviso y vuelve al login	"Comenzamos por el flujo del usuario. En **Crear cuenta** hay dos tipos: visitante y docente; si es docente aparece el campo del colegio. En esta etapa el formulario es **solo visual**: muestra un aviso y lleva al login. El guardado real queda para la prueba 3."

Tiempo	Se ve en el video	Qué dice
2:10–2:40	**Login** con `visitante@aurasacrapp.cl`. Mostrar primero un error (clave mala) y luego el ingreso correcto	"En el **Login** validamos el formulario y el formato del correo, y consultamos a **Room** en una corrutina. La contraseña nunca se guarda en texto plano: se compara el **hash SHA-256**. Si falla, el ViewModel muestra el error. Si sale bien, se guarda la sesión y navegamos a Inicio, sacando Bienvenida y Login del historial."
2:40–3:10	**Inicio**: barra inferior (señalar las 5 pestañas), escribir "maipu" en el buscador, usar filtros por tipo	"**Inicio** lista los recintos con su estado: abierto, cerrado o en celebración. La **barra inferior** tiene cinco pestañas. El buscador funciona **sin importar tildes**: escribo 'maipu' y encuentra 'Maipú'. Los filtros por tipo se combinan con la búsqueda usando `combine` de Flow."
3:10–3:30	**Ficha** de un recinto: datos, horario de la semana, historia, eventos. Tocar *Planificar visita* y mostrar el formulario	"Al tocar un recinto abrimos su **ficha**: tipo, dirección, estado, horario, historia y próximos eventos. Desde ahí va **Planificar visita**, con fecha, hora y cantidad de personas. Mi compañero continúa con rutas, QR y el perfil de encargado."

PERSONA B · 3:30 a 7:00

Tiempo	Se ve en el video	Qué dice
3:30–4:10	Pestaña/botón **Mapa**: tocar un marcador y *Ver ficha*	"Gracias. El **Mapa** muestra cada recinto en una posición calculada a partir de su **latitud y longitud**. Es un mapa esquemático. Al tocar un marcador vemos el recinto y su estado, con un botón hacia la ficha."
4:10–5:00	**Rutas**: lista a detalle de una ruta (paradas, tiempos) a *Crear ruta* (nombre, modo de traslado, marcar recintos)	"En **Rutas** hay recorridos sugeridos con su modo de traslado y tiempo estimado. El detalle muestra las paradas en orden, el tiempo de visita y el de traslado entre paradas. Ese traslado se calcula con la **fórmula de Haversine**, la misma que usa el backend. En **Crear ruta** se elige el modo y los recintos; el botón solo se activa con al menos dos paradas."
5:00–5:30	Pestaña **Agenda** a tocar un evento a detalle a ir a la ficha	"La **Agenda cultural** lista los próximos eventos: conciertos, exposiciones y actividades. Solo aparecen los publicados y que aún no terminan. Al abrir uno vemos su detalle y podemos ir a la ficha del recinto."
5:30–6:15	Pestaña **Escanear**: tocar el botón, apuntar a `qr_01`, aparece "Visita registrada" y se abre la ficha. Luego escanear `qr_99` y mostrar el error. Ir a **Visitas**	"La lectura de **QR** usa el escáner de **ML Kit de Google**, que maneja la cámara a través de Google Play Services. El código tiene el formato `aurasacrapp:recinto:` más el id. El ViewModel valida el formato, comprueba que haya sesión y guarda la **visita en Room**; después abre la ficha. Si el código es de otra página, o el recinto está dado de baja, muestra un error. Las visitas quedan en **Mis visitas**, con fecha y hora."
6:15–6:50	**Cerrar sesión** y entrar con `encargado.capilla@aurasacrapp.cl` a *Administrar mi recinto* a cambiar estado a "En celebración" a volver a Inicio y mostrar que cambió. Pasar rápido por *Editar ficha*, *Publicar evento* y *Guías*	"Por último, el perfil de **encargado**. En **Mi recinto** cambia el estado de apertura, y el cambio se guarda en Room: como las pantallas observan un **Flow**, la lista de Inicio y la ficha se actualizan al instante. Desde el mismo panel están editar la ficha, publicar eventos y ver guías y solicitudes; esos formularios son visuales en esta etapa."
6:50–7:00	**Perfil** a *Cerrar sesión* a vuelve a Bienvenida	"Cerrar sesión limpia el historial de navegación. En la prueba 3 conectaremos la app al backend con Retrofit. Muchas gracias."

Preguntas que probablemente harán (ensayar)

- ¿Qué hace `popUpTo` al hacer login o cerrar sesión?

- ¿Qué diferencia hay entre `StateFlow`, `Flow` y `mutableStateOf`?
- ¿Para qué sirve `viewModelScope.launch`, y por qué Room va en una corrutina?
- ¿Qué hace un DAO? ¿Y `@Entity`, `@PrimaryKey` y `@ForeignKey`?
- ¿Por qué el ViewModel no habla directo con el DAO? (por el Repository)
- ¿Cómo llega el `recintoId` al ViewModel? (`SavedStateHandle` con el argumento de la ruta)
- ¿Por qué no pedimos permiso de cámara? (lo maneja Google Play Services)
- ¿Dónde está el código del QR y cómo se valida? (`EscanearScreen`, `EscanearViewModel`, `CodigoQr`)